Transformer-Koopman: History-Conditioned Low-Rank Transition Modulation with Rollout-Aware Training and Real-Time Predictive Control

Bolin Tao

***Abstract*—Koopman modeling represents nonlinear dynamics as linear evolution in a lifted space whose accuracy depends on the lifting map and the identified operator. Many conventional EDMD and learned-lifting approaches use instantaneous-state observables with a fixed lifted transition, so they neglect short-term temporal correlations and cannot distinguish how the system evolves from different past trajectories. History-aware lifting recovers that information, but one global lifted transition still forces all history-dependent regimes to share a single evolution law. We propose Transformer-Koopman, where a patch-based Transformer encoder maps a finite history window to a structured lifted state and a low-rank history-dependent term modulates a full-rank nominal transition. We show that a one-step-trained history-conditioned operator develops a training–deployment inconsistency under recursive scheduled propagation that spectral stabilization does not repair, whereas training under the deployment rollout restores long-horizon accuracy. The predictor is integrated into a one-iteration real-time-iteration (RTI) MPC that avoids re-running the full encoder at every stage, achieving a $p95$ latency of 120 ms under a 200-ms sampling period and tracking a matched exact scheduled NMPC oracle within 1%. A finite-horizon theory characterizes the rollout-error propagation and the local RTI approximation, supported by empirical diagnostics. Modeling and real-time control results on a 2-state soft robotic platform support the reported claims; a 6-state manipulator and hardware validation are left for future work.**

***Index Terms*—Koopman operator, history-conditioned dynamics, low-rank transition, rollout-aware training, model predictive control, real-time iteration, temporal lifting, Transformer**

I. Introduction

Learning predictive models for nonlinear controlled systems is a central problem in system identification and model-based control [4]–[6]. For controller design, model quality is determined by prediction accuracy and by compatibility with control synthesis; models that are both accurate and structurally suited to control design are therefore of primary interest.

The Koopman framework addresses this by representing nonlinear dynamics as a linear evolution in a lifted space of observable functions [1], [16]. This linear structure enables standard linear prediction and controller design tools [17], [29], making Koopman modeling an attractive paradigm for control. Its effectiveness depends critically on the choice of lifting map.

Many conventional EDMD [8] and learned-lifting approaches [18], [20] use instantaneous-state observables together with a fixed lifted transition, neglecting short-term temporal correlations. Delay-embedding variants enrich the observables with time-shifted measurements [11], [12], and context-aware formulations regenerate the operator from recent data [14], [15]. Enriching the observables with history supplies temporal information that the current measurement alone does not, but it leaves a second question open: a single time-invariant lifted transition still requires every history-dependent operating regime to share one global linear evolution in the learned coordinates, which can place a strong burden on the lifting map when the effective dynamics vary with recent trajectory context. We therefore separate history dependence into two roles—history is encoded into the lifted state, and a structured low-rank correction selectively adjusts a shared nominal transition. Such distinctions can matter more under recursive multi-step prediction.

We propose Transformer-Koopman, a temporal lifting framework with two components. The first component uses a Transformer encoder to map a finite history window into a structured lifted state that retains the measured physical state. The second identifies the Koopman dynamics from the lifted state: rather than requiring all history-dependent regimes to share a single time-invariant operator, we let the history modulate the lifted transition through a low-rank term, so that the transition can adapt to the recent trajectory while the propagation remains linear in the lifted state for a given context. We further show that a history-conditioned operator trained only on encoder-consistent one-step transitions develops a training–deployment inconsistency under recursive scheduled propagation—predicted latent states alter the predicted history, which in turn reschedules the subsequent transition operators—and that spectral stabilization alone does not repair this failure mode. We therefore train the predictor under the same recursively scheduled rollout used at deployment, which restores long-horizon accuracy.

Building on the rollout-trained scheduled predictor, we develop a real-time predictive controller based on a one-iteration RTI-MPC that avoids re-running the full Transformer lifting encoder at every prediction stage, and we establish a finite-horizon theory whose assumptions are assessed by empirical diagnostics on the validation distribution.

The contributions of this paper are summarized as follows:

1) **Low-rank history-conditioned modulation of the lifted transition.** We introduce a Koopman model in which finite-history information forms the lifted repre-

sentation and selectively modulates a full-rank nominal transition through a low-rank history-dependent update, while preserving linear lifted-state propagation for a given context, which is compatible with the scheduled RTI formulation developed later.

2) **Scheduled-rollout inconsistency and deployment-consistent training.** We identify a training–deployment mismatch that arises when a one-step-trained history-conditioned Koopman model is recursively propagated with its context rescheduled from predicted histories, and show that spectral stabilization alone does not repair this failure mode. Training under the same scheduled propagation used at deployment restores long-horizon prediction accuracy, to a level close to recurrent re-encoding.
3) **Real-time predictive deployment.** The rollout-trained scheduled predictor is integrated into a one-iteration RTI-MPC that avoids a full Transformer lifting encoder evaluation at every prediction stage. The resulting controller tracks a matched exact scheduled NMPC oracle within 1%, and a finite-horizon analysis supported by empirical diagnostics characterizes the rollout-error propagation and the local RTI approximation.

II. Related Work

A. History-Based Koopman Representation

Koopman operator theory represents nonlinear dynamics as a linear evolution on the observable functions of the state [1]–[3], enabling the direct application of linear control synthesis to nonlinear systems. Extensions such as DMDc [26], input-aware formulations [27], [28], and Koopman-based MPC [29] established the foundation for Koopman control, and later work added learned representations and performance guarantees [30]–[33]. To learn such models from data, EDMD fits observables from a pre-defined dictionary [8], [9], [34], whereas data-driven methods learn the representations instead [18]–[21]. Many conventional EDMD and learned-lifting approaches use instantaneous-state observables together with a fixed lifted transition, neglecting short-term temporal correlations and limiting long-horizon prediction accuracy.

Two lines relax this restriction. Delay-embedding variants enrich the dictionary with time-shifted measurements or Hankel/HAVOK coordinates [10]–[13], so the operator acts on a history-augmented state, but history enters only through the observables. Transformers model temporal dependencies through self-attention [35], while patch-based tokenization provides an efficient representation of local temporal structure [36]. In Koopman lifting, where the relevant temporal context is unknown a priori, Transformers can incorporate short-term information from past trajectories.

B. Context-Dependent Koopman Dynamics and Recursive Prediction

A history-augmented representation, however, still leaves the transition itself time-invariant. Context-aware and time-varying formulations instead let the effective operator depend on recent data: Koopa computes context-aware operators for non-stationary forecasting [14], and MamKO generates time-varying Koopman matrices from online history [15]. Our formulation separates history dependence into two roles: history is encoded into the lifted state, while a low-rank family identified jointly with the lifting map selectively corrects a shared nominal transition, so that the lifted dynamics remain linear in the lifted state for a given context and provide the conditional-linear structure exploited by the scheduled RTI controller.

Recent work has explored Transformer encoders with Koopman-style latent dynamics for forecasting [37], [38]. In particular, [38] parameterizes the transition as a single low-rank operator with a spectrally constrained diagonal factor on a patch-based Transformer backbone; unlike our full-rank nominal transition modulated by history, it uses a static low-rank transition for forecasting. In contrast, our Transformer is used only for history encoding, while prediction inside the controller is performed by the structured Koopman model. Relatedly, low-rank space-varying transitions have been learned for continuum-robot control, conditioned on the instantaneous state and paired with an infinite-horizon linear-quadratic controller [39].

Multi-step objectives have also been used in learned Koopman models to reduce recursive prediction error [20], [22]–[24], and are standard in neural system identification more broadly [25], so multi-step loss itself is not the focus here. The additional issue in history-conditioned dynamics is that recursive prediction reschedules future transitions from predicted histories, causing both the latent state and the conditioning context to depart from their one-step training distribution. Our rollout-aware training targets this specific deployment mismatch.

C. Real-Time Predictive Control

Two established strategies reduce the online cost of predictive control. The real-time iteration (RTI) performs a single sequential quadratic programming update per sampling instant around the current nominal trajectory [43], [44], and its nominal stability, suboptimality, and sensitivity-based delay reduction have been analyzed [45], [46]. Quasi-linear parameter-varying (quasi-LPV) control instead embeds the nonlinear model into a family of linear models indexed by a scheduling variable, so that freezing or iterating the scheduling trajectory yields a sequence of tractable quadratic programs [47]–[50].

Our controller inherits the one-iteration RTI structure but differs in where the scheduling signal comes from. Quasi-LPV MPC reads the scheduling variable from the state or velocity, using the previous solution or a planned reference; here the transition is scheduled by a context that a learned encoder extracts from a history window, and during prediction the history is itself produced by the predictor, so the scheduling trajectory is endogenous and cannot be read off the measured state. The context must therefore be re-evaluated along the rolled-out history rather than assumed, and the full Transformer lifting encoder need not be re-run at every prediction stage, letting the single-iteration controller fit inside the sampling period.

III. Preliminaries and Problem Formulation

In this section, we review the controlled Koopman formulation, introduce finite-history lifting, and state the data-driven learning problem considered in this work. The specific Transformer realization and the history-conditioned low-rank transition are developed in Section IV.

A. Koopman Modeling and Temporal Lifting

Consider a discrete-time controlled nonlinear system

$$\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k), \tag{1}$$

where $\mathbf{x}_k \in \mathbb{R}^n$ is the measured state, $\mathbf{u}_k \in \mathbb{R}^m$ is the control input, and $F : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ is an unknown nonlinear transition map. Here $\mathbf{x}_k$ denotes the measured coordinates used by the predictor; finite history is introduced below when these coordinates are not Markov-complete. State-input trajectories $\{(\mathbf{x}_k, \mathbf{u}_k)\}$ are assumed to be available for data-driven modeling.

The Koopman framework represents nonlinear dynamics through the evolution of observable functions rather than directly through the state [1], [3], [7], [16]. For a fixed control input $\mathbf{u}$, the input-parameterized Koopman operator $\mathcal{K}_\mathbf{u}$ acts on a scalar observable $\varphi : \mathbb{R}^n \to \mathbb{R}$ as

$$(\mathcal{K}_\mathbf{u}\varphi)(\mathbf{x}) = \varphi(F(\mathbf{x}, \mathbf{u})). \tag{2}$$

For each fixed $\mathbf{u}$, $\mathcal{K}_\mathbf{u}$ is linear in the observable:

$$\mathcal{K}_\mathbf{u}(\alpha\varphi + \beta\psi) = \alpha\mathcal{K}_\mathbf{u}\varphi + \beta\mathcal{K}_\mathbf{u}\psi. \tag{3}$$

The Koopman operator is generally infinite-dimensional, so practical computation relies on a finite-dimensional approximation. In EDMD, one selects d scalar observables and stacks them into the lifting map

$$\mathbf{z}_k = g(\mathbf{x}_k) = [g_1(\mathbf{x}_k), \dots, g_d(\mathbf{x}_k)]^\top \in \mathbb{R}^d. \tag{4}$$

The lifted dynamics are then approximated by a linear system $\mathbf{z}_{k+1} \approx \mathbf{A}\mathbf{z}_k + \mathbf{B}\mathbf{u}_k$, with $\mathbf{A}$ and $\mathbf{B}$ identified from data. Instantaneous-state lifting may miss temporal information that matters for prediction, particularly when the measured state does not by itself characterize the underlying dynamics.

To incorporate recent trajectory information, we construct the lifting map from a history window of length P:

$$\mathbf{X}_k = [\mathbf{x}_{k-P+1}, \dots, \mathbf{x}_k]^\top \in \mathbb{R}^{P\times n}, \tag{5}$$

and define

$$\mathbf{z}_k = g_\theta(\mathbf{X}_k) \in \mathbb{R}^d. \tag{6}$$

The resulting temporal lifting incorporates recent trajectory information, with the nominal lifted dynamics written as

$$\mathbf{z}_{k+1} = \mathbf{A}\mathbf{z}_k + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k, \tag{7}$$

where $\mathbf{w}_k$ denotes the lifted-space approximation residual. In this work, g_θ is implemented using a Transformer encoder. Temporal lifting introduces recent trajectory information into the latent state; the remaining question is how to accommodate history-dependent variations in the lifted dynamics without abandoning a control-compatible structure.

B. Prediction Problem and Design Requirements

We assume that the available data consist of recorded state–input trajectories

$$\mathcal{D} = \{\tau^{(j)}\}_{j=1}^{N_{\mathrm{tr}}}, \qquad \tau^{(j)} = \{(\mathbf{x}_t^{(j)}, \mathbf{u}_t^{(j)})\}_{t=0}^{T_j}. \tag{8}$$

where N_{tr} denotes the number of recorded trajectories and T_j the number of samples in the j-th trajectory. For each admissible time index k, the state-history window $\mathbf{X}_k$ is defined by (5), and the aligned past-input history is

$$\mathbf{U}_k^{\mathrm{in}} = [\mathbf{u}_{k-P+1}^\top, \dots, \mathbf{u}_{k-1}^\top]^\top \in \mathbb{R}^{(P-1)m}. \tag{9}$$

Because the measured state need not form a complete Markov state, the predictor must exploit finite history. However, using history only in the lifting map does not by itself remove the restriction imposed by a single global lifted transition: all history-dependent latent states would still be required to evolve under the same linear operator. Conversely, replacing the lifted dynamics with an unstructured nonlinear predictor would weaken the linear structure that makes Koopman models attractive for predictive control.

A control-oriented predictor should therefore exploit finite history to represent temporal information that the instantaneous measurement does not provide, allow the lifted transition to adapt to recent trajectory context while retaining linear propagation for a fixed context, and remain accurate when its own predicted histories are recursively fed back during multi-step prediction.

A predictor satisfying these requirements takes the form of a state-preserving temporal lifting

$$\mathbf{z}_k = g_\theta(\mathbf{X}_k), \qquad \mathbf{C}\mathbf{z}_k = \mathbf{x}_k, \tag{10}$$

together with a history-conditioned lifted predictor of the form

$$\hat{\mathbf{z}}_{k+1} = \mathcal{A}_\psi(\mathbf{X}_k, \mathbf{U}_k^{\mathrm{in}})\,\mathbf{z}_k + \mathbf{B}\mathbf{u}_k, \qquad \hat{\mathbf{x}}_{k+1} = \mathbf{C}\hat{\mathbf{z}}_{k+1}, \tag{11}$$

where the current control input enters through a context-independent linear channel. Section IV instantiates this predictor.

IV. History-Conditioned Transformer–Koopman Modeling

We instantiate the predictor of Section III-B using a patch-based temporal lifting, a history-conditioned low-rank transition, and deployment-consistent scheduled training.

A. Temporal Lifting and History-Conditioned Transition

We now introduce the proposed temporal lifting architecture.

Given the history window $\mathbf{X}_k \in \mathbb{R}^{P\times n}$, we partition it into $N_p = P/p$ contiguous non-overlapping temporal patches of length p, where p is assumed to divide P. The ith patch is denoted by

$$\mathbf{X}_k^{(i)} \in \mathbb{R}^{p\times n}, \qquad i = 1, \dots, N_p. \tag{12}$$

Each patch is then vectorized, meaning it is flattened (or "unrolled") into a single vector, as

$$\mathbf{q}_k^{(i)} = \mathrm{vec}(\mathbf{X}_k^{(i)}) \in \mathbb{R}^{pn}, \tag{13}$$

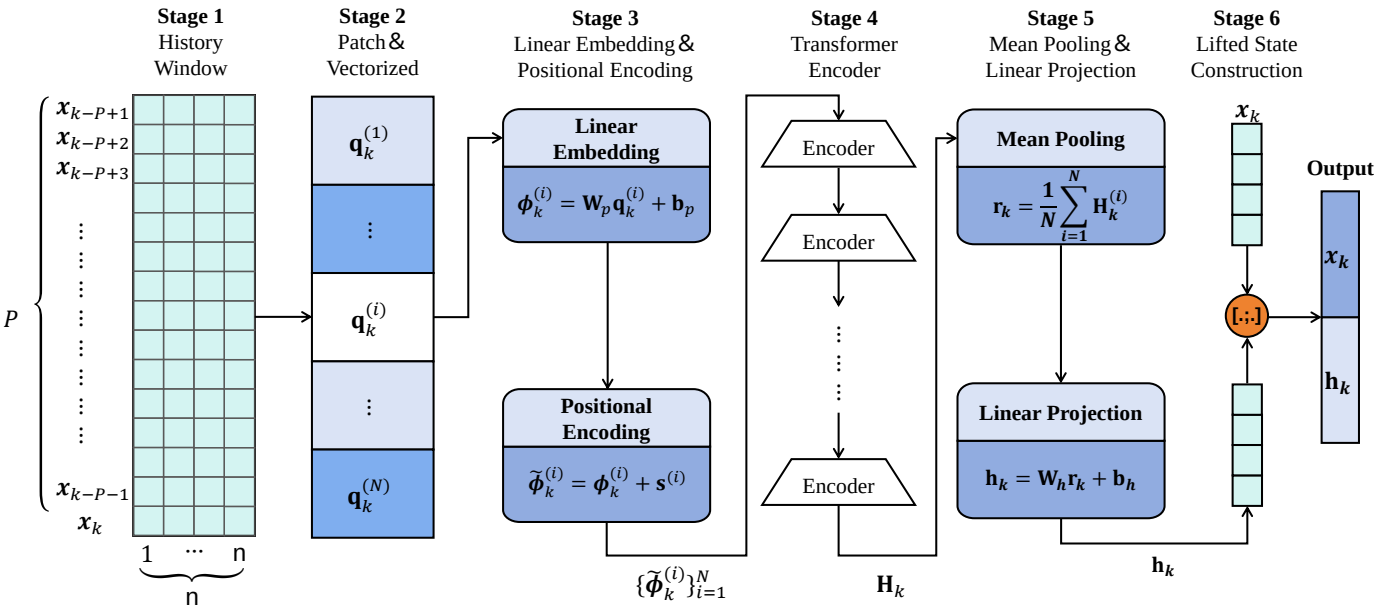


Fig. 1. Overall prediction pipeline of the proposed Transformer-Koopman framework. Given a history window $\mathbf{X}_k$ and the current control input $\mathbf{u}_k$, the Transformer encoder extracts a history feature $\mathbf{h}_k$, which is concatenated with the current state $\mathbf{x}_k$ to form the lifted state $\mathbf{z}_k = [\mathbf{x}_k; \mathbf{h}_k] \in \mathbb{R}^d$. The Koopman linear dynamics $\hat{\mathbf{z}}_{k+1} = \mathbf{A}(\mathbf{c}_k)\mathbf{z}_k + \mathbf{B}\mathbf{u}_k$ propagate the lifted state forward, and the predicted next physical state $\hat{\mathbf{x}}_{k+1} = \mathbf{C}\hat{\mathbf{z}}_{k+1}$ is recovered through a fixed linear projection.

which is then mapped into the Transformer embedding space through a shared linear layer,

$$\boldsymbol{\phi}_k^{(i)} = \mathbf{W}_p \mathbf{q}_k^{(i)} + \mathbf{b}_p \in \mathbb{R}^{d_{\text{model}}}, \tag{14}$$

where $\mathbf{W}_p \in \mathbb{R}^{d_{\text{model}} \times pn}$ and $\mathbf{b}_p \in \mathbb{R}^{d_{\text{model}}}$. This patch-based tokenization reduces the sequence length from P to $N_p = P/p$ while preserving local temporal structure within each patch.

The token sequence is augmented with sinusoidal positional encoding [35] to incorporate temporal order, and is then processed by an L-layer Transformer encoder:

$$\tilde{\boldsymbol{\phi}}_k^{(i)} = \boldsymbol{\phi}_k^{(i)} + \mathbf{s}^{(i)}, \tag{15}$$

and the resulting token sequence is encoded as

$$\mathbf{H}_k = \text{TransformerEncoder}\left(\{\tilde{\boldsymbol{\phi}}_k^{(i)}\}_{i=1}^{N_p}\right) \in \mathbb{R}^{N_p \times d_{\text{model}}}, \tag{16}$$

where $\mathbf{H}_k^{(i)} \in \mathbb{R}^{d_{\text{model}}}$ denotes the contextualized representation of the ith patch.

The Transformer encoder outputs $\mathbf{H}_k \in \mathbb{R}^{N_p \times d_{\text{model}}}$. We then apply mean pooling over the token dimension to obtain a single representation:

$$\mathbf{r}_k = \frac{1}{N_p} \sum_{i=1}^{N_p} \mathbf{H}_k^{(i)} \in \mathbb{R}^{d_{\text{model}}}. \tag{17}$$

After that, the pooled representation is projected to the history-feature space,

$$\mathbf{h}_k = \mathbf{W}_h \mathbf{r}_k + \mathbf{b}_h \in \mathbb{R}^{d-n}, \tag{18}$$

where $\mathbf{W}_h \in \mathbb{R}^{(d-n) \times d_{\text{model}}}$ and $\mathbf{b}_h \in \mathbb{R}^{d-n}$.

The lifted state is defined as

$$\mathbf{z}_k = [\mathbf{x}_k; \mathbf{h}_k] \in \mathbb{R}^d, \tag{19}$$

where the first n components of $\mathbf{z}_k$ are the original system state $\mathbf{x}_k$, and the remaining $d - n$ components correspond to the temporal features $\mathbf{h}_k$. This structure allows for exact recovery of the original system state through the fixed projection

$$\mathbf{x}_k = \mathbf{C}\mathbf{z}_k, \qquad \mathbf{C} = [\mathbf{I}_n \ \mathbf{0}], \tag{20}$$

The overall architecture of the proposed framework is illustrated in Fig. 1.

Although the temporal lifting provides different latent representations for different histories, a fixed transition $\mathbf{A}$ still requires all such history-dependent latent states to evolve under one global linear operator. To relax this additional restriction without discarding the conditional-linear structure, we decompose the transition into a shared nominal component and a structured history-dependent correction.

Definition 1 (History-Conditioned Low-Rank Koopman Family)**.** *Let the history window be* $\mathbf{X}_k$ (5)*, and let* $\mathbf{U}_k^{\text{in}} = [\mathbf{u}_{k-P+1}^\top, \ldots, \mathbf{u}_{k-1}^\top]^\top$ *denote the input history over the same window. Define the lifted state and the context vector*

$$\mathbf{z}_k = g_\theta(\mathbf{X}_k) = [\mathbf{x}_k; \mathbf{h}_k], \qquad \mathbf{c}_k = C_\phi(\mathbf{X}_k, \mathbf{U}_k^{\text{in}}) \in \mathbb{R}^{d_c}, \tag{21}$$

where C_ϕ *is a context network. The lifting map uses the state history to construct a state-preserving representation, whereas the context map additionally uses the past inputs to condition the effective transition on how the current dynamical condition was reached. The lifted dynamics are modeled as*

$$\mathbf{z}_{k+1} = \mathbf{A}(\mathbf{c}_k)\, \mathbf{z}_k + \mathbf{B}\, \mathbf{u}_k, \tag{22}$$

with the history-conditioned *transition*

$$\mathbf{A}(\mathbf{c}) = \mathbf{A}_0 + \mathbf{U}\, \text{diag}\big(\gamma_\psi(\mathbf{c})\big)\, \mathbf{V}^\top, \qquad \text{rank}\big[\mathbf{A}(\mathbf{c}) - \mathbf{A}_0\big] \le r. \tag{23}$$

Here $\mathbf{A}_0 \in \mathbb{R}^{d\times d}$, $\mathbf{U},\mathbf{V} \in \mathbb{R}^{d\times r}$, $\gamma_\psi : \mathbb{R}^{d_c} \to \mathbb{R}^r$, *and* $\mathbf{B} \in \mathbb{R}^{d\times m}$ *is* context-independent *(not context-modulated). Throughout,* $r = 2$.

$\mathbf{A}_0$ represents the shared nominal lifted dynamics, while the low-rank term provides a structured context-dependent correction. Context dependence is confined to $\mathbf{A}(\mathbf{c})$; $\mathbf{B}$ is a single global matrix shared across all contexts, so the current input always enters through the same affine channel. The resulting model is a history-conditioned parameter-varying lifted system that remains linear in $(\mathbf{z}_k, \mathbf{u}_k)$ for a fixed context, and the fixed projection (20) recovers the physical state exactly. Section VII-A shows that the modulation must be refined jointly with the temporal representation and the nominal transition, since training it alone degrades deployment accuracy by more than an order of magnitude.

B. Training Procedure

The model of Definition 1 has three trainable parts: the lifting-encoder parameters θ that produce the lifted state $\mathbf{z}_k$, the context parameters ϕ that produce $\mathbf{c}_k = C_\phi(\mathbf{X}_k, \mathbf{U}_k^{\text{in}})$, and the dynamics parameters $\mathbf{A}_0$, $\mathbf{U}$, $\mathbf{V}$, and γ_ψ. The input matrix $\mathbf{B}$ is context-independent: a single global matrix is shared across all contexts and is optimized jointly with the other parameters. The projection $\mathbf{C}$ is structurally fixed throughout training, as in Definition 1. Training operates on length-H_R segments cut from single trajectories: each segment provides the history window $\mathbf{X}_k$, the past-input history $\mathbf{U}_k^{\text{in}}$, the future inputs $\mathbf{u}_{k:k+H_R-1}$, and the future states $\mathbf{x}_{k+1:k+H_R}$ as prediction targets, and segments do not cross trajectory boundaries.

The history-conditioned transition introduces an additional deployment issue. During one-step training the context is evaluated on measured histories, whereas during recursive prediction future contexts are evaluated on predicted histories, so prediction errors alter the scheduling signal itself. Training therefore proceeds in two stages, which we treat as an optimization schedule rather than a requirement of the model: a nominal one-step pretraining stage that provides a well-scaled initialization, followed by scheduled-rollout fine-tuning that jointly refines the full history-conditioned family on the *propagated*-latent distribution that arises at deployment, by minimizing the rollout-aware scheduled objective $\mathcal{L}_R$ of Definition 2.

Stage 0 (nominal one-step pretraining). We initialize the encoder g_θ and a time-invariant base operator $(\mathbf{A}_0, \mathbf{B})$ by minimizing the one-step physical prediction error over the encoder-generated latent distribution,

$$\mathcal{L}_{\text{init}} = \sum_k \left\| \mathbf{C}\big(\mathbf{A}_0\, \mathbf{z}_k + \mathbf{B}\, \mathbf{u}_k\big) - \mathbf{x}_{k+1} \right\|_2^2, \qquad \mathbf{z}_k = g_\theta(\mathbf{X}_k). \tag{24}$$

This stage provides a well-scaled lifted representation and a data-fitted nominal base operator; it is the only stage optimized exclusively on the encoder-consistent one-step latent distribution, and it corresponds to the fixed (time-invariant) Transformer-Koopman baseline. It serves as an optimization initialization: Section VII-A shows that omitting it does not degrade deployment accuracy.

Definition 2 (Rollout-Aware Scheduled Objective)**.** *With the scheduled rollout* $\mathbf{z}^P$ *(latent propagated, context re-scheduled from the decoded history) and uniform weights* $w_i = 1/H_R$*,*

$$\mathcal{L}_R = \sum_k \sum_{i=1}^{H_R} w_i \left\| \mathbf{C}\mathbf{z}_{k+i}^P - \mathbf{x}_{k+i} \right\|_2^2. \tag{25}$$

At deployment the transition consumes *propagated* latents $\mathbf{z}^P$, whose distribution differs from the encoder-generated one-step distribution on which Stage 0 is fitted. The rollout objective is an algorithmic training mechanism rather than a theoretical guarantee: no statement in Section V uses $\mathcal{L}_R$, and Section V analyzes rollout consistency independently through finite-horizon error propagation.

Stage 1 (scheduled-rollout fine-tuning). We then activate the low-rank context modulation $\mathbf{A}(\mathbf{c}_k) = \mathbf{A}_0 + \mathbf{U}\,\mathrm{diag}(\gamma_\psi(\mathbf{c}_k))\,\mathbf{V}^\top$ and jointly refine θ, ϕ, $\mathbf{A}_0$, $\mathbf{U}$, $\mathbf{V}$, γ_ψ, and the shared input matrix $\mathbf{B}$ by minimizing the rollout-aware scheduled objective $\mathcal{L}_R$ of Definition 2 (25), in which the latent is propagated through $\mathbf{A}(\mathbf{c}_{k+i-1})$ while the context is *re-scheduled* from the history that the predictor itself has rolled out, $\mathbf{c}_{k+i-1} = C_\phi(\hat{\mathbf{X}}_{k+i-1}, \mathbf{U}_{k+i-1}^{\text{in}})$ (Section V-B). Only the projection $\mathbf{C}$ is structurally fixed. We use horizon $H_R = 20$ with uniform weights w_i.

Minimizing $\mathcal{L}_R$ trains the operator on the propagated-latent distribution that arises at deployment, rather than on the encoder-generated one-step distribution—precisely the distribution shift analyzed in Section V-B. No closed-form EDMD refit is applied after the transition is activated, since the regression target is no longer a single operator shared across all contexts. Spectral stabilization is not relied upon: Remark 2 shows that constraining $\rho(\mathbf{A}(\mathbf{c})) < 1$ does not restore scheduled-rollout accuracy.

The modulation is parameterized with a bounded budget so that the lifted dynamics remain well-conditioned: the columns of $\mathbf{U}$ and $\mathbf{V}$ are unit-norm and γ_ψ is bounded elementwise, so $\|\mathbf{A}(\mathbf{c}) - \mathbf{A}_0\|_2$ is bounded uniformly in $\mathbf{c}$. Throughout, $r = 2$, a small rank chosen so that the modulation remains a bounded low-rank correction of the nominal transition.

For the reported predictor, the checkpoint with the lowest validation rollout loss over the fine-tuning stage is retained, and no spectral penalty is applied; the ablation of Section VII-A instead trains each variant for a fixed number of epochs without checkpoint selection. The complete procedure is summarized in Algorithm 1.

C. Modeling Error Characterization

The history-conditioned model (22) is only approximate, due to finite-dimensional Koopman truncation and encoder approximation error. We define the one-step modeling residual at the encoder-consistent latent as

$$\mathbf{w}_k = \mathbf{z}_{k+1} - \big(\mathbf{A}(\mathbf{c}_k)\mathbf{z}_k + \mathbf{B}\mathbf{u}_k\big), \tag{26}$$

where $\mathbf{z}_{k+1} = g_\theta(\mathbf{X}_{k+1})$ and $\mathbf{z}_k = g_\theta(\mathbf{X}_k)$. Here $\mathbf{w}_k$ is evaluated on recorded histories and measures one-step data consistency. It is distinct from the rollout-consistency defect ϵ_k of Proposition 2, which is evaluated on predictor-generated histories. A small one-step residual alone does not imply

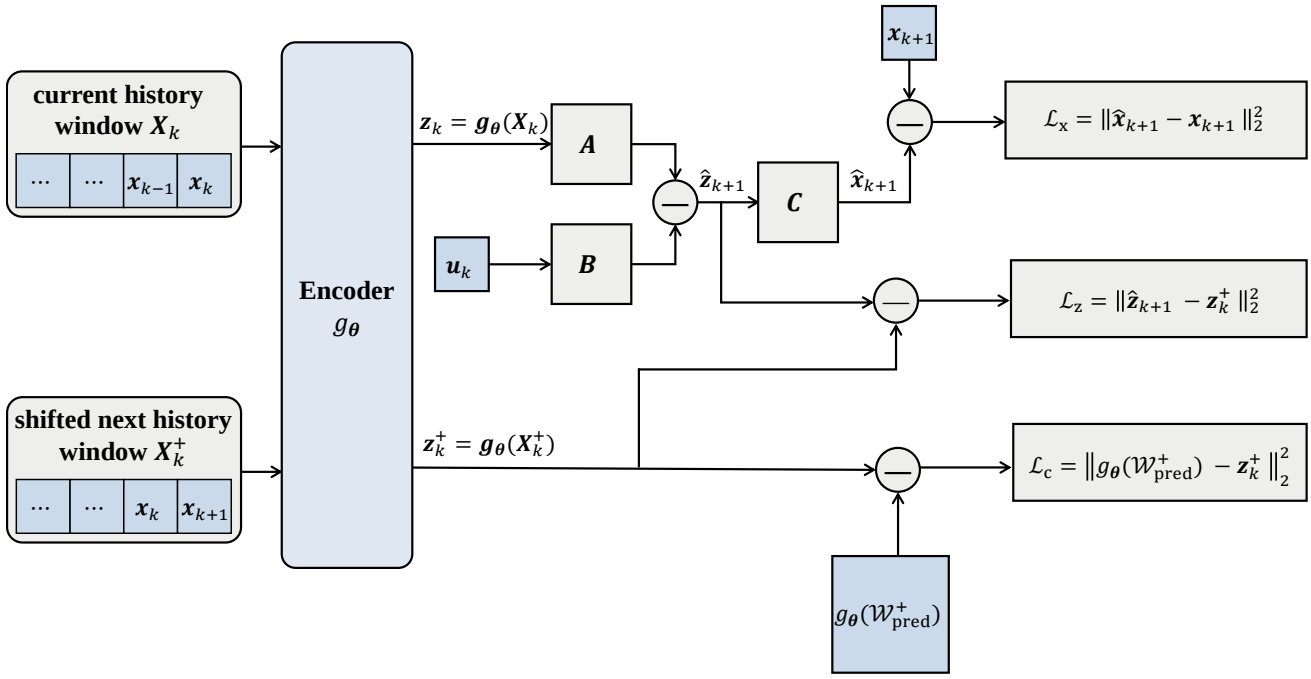


Fig. 2. One-step lifting and prediction (nominal pretraining, Stage 0). The history window $\mathbf{X}_k$ and its shift $\mathbf{X}_k^+$ are each encoded by g_θ to produce $\mathbf{z}_k$ and $\mathbf{z}_k^+$. The time-invariant base operator $(\mathbf{A}_0, \mathbf{B})$ produces the one-step prediction $\hat{\mathbf{z}}_{k+1} = \mathbf{A}_0\mathbf{z}_k + \mathbf{B}\mathbf{u}_k$, from which $\hat{\mathbf{x}}_{k+1}$ is recovered via $\mathbf{C}$. This one-step coupling is used to initialize the encoder and base operator, supervised by the one-step loss (24). Stage 1 replaces $\mathbf{A}_0$ by the history-conditioned $\mathbf{A}(\mathbf{c})$ and jointly refines the full model on the rollout-aware scheduled objective (Algorithm 1).

Algorithm 1 Transformer-Koopman Training Algorithm

Input: Trajectory dataset $\mathcal{D}$ (8)
History length P; patch length p; lifted dimension d; rank r; rollout horizon H_R
Rollout weights $w_1, \ldots, w_{H_R}$; epochs E_0 (nominal pretraining), E_1 (scheduled-rollout fine-tuning)
Output: Lifting encoder θ^*; context ϕ^*; transition parameters $\mathbf{A}_0^*, \mathbf{U}^*, \mathbf{V}^*, \gamma_\psi^*$; input matrix $\mathbf{B}^*$
Stage 0 (nominal one-step pretraining): Initialize θ and $(\mathbf{A}_0, \mathbf{B})$; for E_0 epochs, minimize the one-step loss (24) over $(\theta, \mathbf{A}_0, \mathbf{B})$.
Stage 1 (scheduled-rollout fine-tuning): Activate $\mathbf{A}(\mathbf{c}) = \mathbf{A}_0 + \mathbf{U}\,\mathrm{diag}(\gamma_\psi(\mathbf{c}))\,\mathbf{V}^\top$; keep only $\mathbf{C}$ structurally fixed.
for epoch $\ell = 1$ **to** E_1 **do**
  for each mini-batch of length-H_R segments $\mathcal{B}$ drawn from $\mathcal{D}$ **do**
    Roll out the latent over H_R steps, re-scheduling the context $\mathbf{c}_{k+i} = C_\phi(\hat{\mathbf{X}}_{k+i}, \mathbf{U}_{k+i}^{\mathrm{in}})$ from the rolled-out history.
    Form the scheduled objective $\mathcal{L}_R$ (25).
    Update $(\theta, \phi, \mathbf{A}_0, \mathbf{U}, \mathbf{V}, \gamma_\psi, \mathbf{B})$ via gradient descent.
  end for
end for
return $\theta^*, \phi^*, \mathbf{A}_0^*, \mathbf{U}^*, \mathbf{V}^*, \gamma_\psi^*, \mathbf{B}^*$

accurate recursive rollout; the propagation of this residual is analyzed in Section V-B and motivates the rollout-aware objective (25).

V. Finite-Horizon Consistency Analysis

This section develops a finite-horizon theory for the history-conditioned low-rank Koopman family introduced in Section IV, i.e. $\mathbf{z}_{k+1} = \mathbf{A}(\mathbf{c}_k)\mathbf{z}_k + \mathbf{B}\mathbf{u}_k$ with $\mathbf{A}(\mathbf{c}) = \mathbf{A}_0 + \mathbf{U}\,\mathrm{diag}(\gamma_\psi(\mathbf{c}))\mathbf{V}^\top$ (23). The analysis is deliberately restricted to *finite-horizon prediction consistency* and the *local* accuracy of the real-time controller; we do not claim global asymptotic closed-loop stability, common Lyapunov functions, terminal invariant sets, or recursive feasibility under learned uncertainty (see the scope remark below). The assumptions and quantities of this section are empirically assessed on the validation distribution in Section VII-B; they are diagnostics, not worst-case certificates over Ω.

A. Finite-History Markovization

Because the lifted state $\mathbf{z}_k$ is a function of a finite window, the predictor can be recast as a first-order system on an augmented state.

Proposition 1 (Finite-History Predictor Markovization)**.** *Define the augmented state*

$$\boldsymbol{\xi}_k = \mathrm{col}\big(\mathbf{x}_{k-P+1:k},\ \mathbf{u}_{k-P+1:k-1},\ \mathbf{z}_k\big) \in \mathbb{R}^{Pn+(P-1)m+d}. \tag{27}$$

Under the predictor's own closed loop—i.e. the newest fed-back output is the decoded $\hat{\mathbf{x}}_{k+1} = \mathbf{C}\mathbf{z}_{k+1}$—there exists a deterministic map $\mathbf{F}_\Theta$ such that

$$\boldsymbol{\xi}_{k+1} = \mathbf{F}_\Theta(\boldsymbol{\xi}_k, \mathbf{u}_k). \tag{28}$$

Proof. The window update is a deterministic shift: append $(\hat{\mathbf{x}}_{k+1}, \mathbf{u}_k)$ and drop the oldest entry. The appended output

is $\hat{\mathbf{x}}_{k+1} = \mathbf{C}\mathbf{z}_{k+1}$ with $\mathbf{z}_{k+1} = \mathbf{A}(\mathbf{c}_k)\mathbf{z}_k + \mathbf{B}\mathbf{u}_k$ and $\mathbf{c}_k = C_\phi(\mathbf{X}_k, \mathbf{U}_k^{\text{in}})$. Since $\mathbf{X}_k$ and $\mathbf{U}_k^{\text{in}}$ are sub-vectors of $\boldsymbol{\xi}_k$, every argument of the update is a function of $(\boldsymbol{\xi}_k, \mathbf{u}_k)$ alone; composing the shift, context map, transition, and decoder yields a single deterministic $\mathbf{F}_\Theta$. □

Remark 1. *Proposition 1 holds* only *for the internal rollout with decoded feedback. If the true measurement $\mathbf{x}_{k+1}$ is fed back, the update also depends on an exogenous input and is not a function of $\boldsymbol{\xi}_k$ alone.*

B. Latent Rollout Inconsistency and Its Propagation

We now characterize the discrepancy that arises when the latent state is propagated recursively instead of being re-encoded at every step.

Definition 3 (Latent Rollout Inconsistency)**.** *Let $\hat{\mathbf{X}}_{k+i}$ be the history produced by the predictor's own rollout and let $\mathbf{c}_{k+i} = C_\phi(\hat{\mathbf{X}}_{k+i}, \mathbf{U}_{k+i}^{\text{in}})$ denote the scheduled context re-computed from it. Define the encoder-consistent latent and the propagated latent,*

$$\mathbf{z}_{k+i}^E = g_\theta(\hat{\mathbf{X}}_{k+i}), \qquad \mathbf{z}_{k+i+1}^P = \mathbf{A}(\mathbf{c}_{k+i})\mathbf{z}_{k+i}^P + \mathbf{B}\mathbf{u}_{k+i}, \tag{29}$$

and the normalized rollout defect

$$d_{\mathcal{M},k+i} = \frac{\left\|\mathbf{z}_{k+i}^P - \mathbf{z}_{k+i}^E\right\|_2}{\left\|\mathbf{z}_{k+i}^E\right\|_2 + \epsilon_0}, \tag{30}$$

with $\epsilon_0 = 10^{-6}$ a fixed numerical floor. We also use the absolute defect $d_{k+i}^{\text{abs}} = \|\mathbf{z}_{k+i}^P - \mathbf{z}_{k+i}^E\|_2$, which is the quantity for which Proposition 2 provides an upper bound.

Proposition 2 (Latent Rollout Error Propagation)**.** *Let $\mathbf{z}_k^P$ denote the recursively propagated latent state and $\mathbf{z}_k^E = g_\theta(\hat{\mathbf{X}}_k)$ the encoder-consistent latent state, and let $d_k \equiv d_k^{\text{abs}} = \|\mathbf{z}_k^P - \mathbf{z}_k^E\|_2$. Define the encoder-consistent one-step defect*

$$\epsilon_k = \left\|\mathbf{A}(\mathbf{c}_k)\mathbf{z}_k^E + \mathbf{B}\mathbf{u}_k - \mathbf{z}_{k+1}^E\right\|_2. \tag{31}$$

Then $d_{k+1} \le \|\mathbf{A}(\mathbf{c}_k)\|_2\, d_k + \epsilon_k$, and for any horizon H,

$$d_{k+H} \le \left\|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\right\|_2 d_k + \sum_{i=0}^{H-1} \left\|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_{k+i+1}\right\|_2 \epsilon_{k+i}, \tag{32}$$

where $\mathbf{A}_{k+j} = \mathbf{A}(\mathbf{c}_{k+j})$ is the transition at the scheduled context $\mathbf{c}_{k+j} = C_\phi(\hat{\mathbf{X}}_{k+j}, \mathbf{U}_{k+j}^{\text{in}})$ along the same rollout. If the rollout starts from an encoder-consistent latent ($\mathbf{z}_k^P = \mathbf{z}_k^E$, hence $d_k = 0$), the first term vanishes and the bound reduces to the second sum alone. The bound is a trajectory-wise recursion: both ϵ_k and the transition matrices $\mathbf{A}_{k+j}$ are evaluated along the same rollout, so they depend on the realized trajectory rather than being a priori constants.

Proof. Let $\mathbf{e}_k = \mathbf{z}_k^P - \mathbf{z}_k^E$ denote the latent error. Subtracting the encoder-consistent update from the propagation rule $\mathbf{z}_{k+1}^P = \mathbf{A}(\mathbf{c}_k)\mathbf{z}_k^P + \mathbf{B}\mathbf{u}_k$ gives the linear recursion

$$\mathbf{e}_{k+1} = \mathbf{A}(\mathbf{c}_k)\mathbf{e}_k + \mathbf{r}_k, \qquad \mathbf{r}_k^z = \mathbf{A}(\mathbf{c}_k)\mathbf{z}_k^E + \mathbf{B}\mathbf{u}_k - \mathbf{z}_{k+1}^E,$$

where $\|\mathbf{r}_k^z\|_2 = \epsilon_k$. Taking norms and using the triangle inequality and submultiplicativity yields the one-step bound $d_{k+1} \le \|\mathbf{A}(\mathbf{c}_k)\|_2\, d_k + \epsilon_k$. Unrolling the recursion over H steps gives

$$\mathbf{e}_{k+H} = \mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\,\mathbf{e}_k + \sum_{i=0}^{H-1} \mathbf{A}_{k+H-1}\cdots\mathbf{A}_{k+i+1}\,\mathbf{r}_{k+i}^z,$$

and taking norms, with $\|\mathbf{e}_k\|_2 = d_k$ and $\|\mathbf{r}_{k+i}^z\|_2 = \epsilon_{k+i}$, gives (32) (the empty product at $i = H-1$ is the identity). Since $\|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\|_2 \le \prod_{j=0}^{H-1}\|\mathbf{A}_{k+j}\|_2$, this matrix-product form is at least as tight as the product-of-norms form. □

Equation (32) shows that the long-horizon inconsistency is governed by two factors: the local one-step defect ϵ_k and the *amplification of the time-varying transition products*.

Remark 2 (Why spectral stabilization is insufficient)**.** *Enforcing $\rho(\mathbf{A}(\mathbf{c}_k)) < 1$ for all k does* not *imply a uniform contraction of the time-varying products:*

$$\rho(\mathbf{A}_k) < 1 \;\; \forall k \quad \not\Rightarrow \quad \|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\|_2 \le c < 1, \tag{33}$$

which is the classical spectral-radius/joint-spectral-radius gap for non-commuting matrices [40]–[42]. This explains our experimental observation (Section VII-A) that constraining $\rho(\mathbf{A}) \le 0.99$ fails to restore scheduled-rollout accuracy, whereas changing the training objective does.

C. Finite-Horizon Prediction Error Bound

To state the bound we compare the learned predictor with a *reference augmented sequence* built directly from the recorded observations, rather than postulating a deterministic reference transition for the plant. Let

$$\boldsymbol{\xi}_k^\star = \text{col}\big(\mathbf{x}_{k-P+1:k},\, \mathbf{u}_{k-P+1:k-1},\, g_\theta(\mathbf{X}_k)\big) \tag{34}$$

denote the reference augmented state built from the true observations and the encoder-consistent latent, and let $\boldsymbol{\xi}_{k+1}^\star$ be its successor along the recorded trajectory. The reference sequence consists of measured quantities; no deterministic reference map is assumed to generate it, so the finite observed history is never asserted to Markovize the underlying plant. The learned augmented transition $\mathbf{F}_\Theta$ is the map of Proposition 1, which propagates the latent and feeds back the decoded output $\hat{\mathbf{x}} = \mathbf{C}\mathbf{z}^P$. We therefore measure the learned model by its one-step discrepancy on the reference sequence,

$$\delta_k^\star = \left\|\boldsymbol{\xi}_{k+1}^\star - \mathbf{F}_\Theta(\boldsymbol{\xi}_k^\star, \mathbf{u}_k)\right\|_2. \tag{35}$$

This quantity bundles the physical modeling error, the finite-history truncation error, the learned transition error, and the difference between the encoder-consistent and the propagated latent updates.

Assumption 1 (Operating-set reference discrepancy)**.** *There exist a compact operating set Ω and a finite constant $\epsilon \ge 0$ such that, along the reference sequence and at every step of the considered horizon, the one-step reference discrepancy* (35) *satisfies $\delta_k^\star \le \epsilon$, equivalently*

$$\left\|\boldsymbol{\xi}_{k+1}^\star - \mathbf{F}_\Theta(\boldsymbol{\xi}_k^\star, \mathbf{u}_k)\right\|_2 \le \epsilon. \tag{36}$$

We additionally require that the reference and predicted trajectories remain in Ω over the considered horizon; this condition is used explicitly in Theorem 1.

Assumption 2 (Lipschitz continuity of the learned transition)**.** *There exists a finite constant L_{F} such that*

$$\|\mathbf{F}_\Theta(\boldsymbol{\xi}_1, \mathbf{u}) - \mathbf{F}_\Theta(\boldsymbol{\xi}_2, \mathbf{u})\|_2 \le L_{\mathrm{F}} \|\boldsymbol{\xi}_1 - \boldsymbol{\xi}_2\|_2 \tag{37}$$

for all $(\boldsymbol{\xi}_1, \mathbf{u}), (\boldsymbol{\xi}_2, \mathbf{u}) \in \Omega$. No contraction is assumed: L_{F} need not be smaller than one. This bound holds whenever the context map C_ϕ, the modulation γ_ψ, and the map $\mathbf{c} \mapsto \mathbf{A}(\mathbf{c})$ are Lipschitz on Ω and the lifted state is bounded on Ω; we take these as standing regularity conditions on the learned components. They hold for the networks used here: the context map and the modulation use tanh *activations, the patch embedding, pooling and projection layers are affine, and the Transformer feed-forward blocks are piecewise affine with ReLU units, so $\mathbf{F}_\Theta$ is Lipschitz and piecewise smooth rather than globally $\mathcal{C}^2$.*

Assumption 3 (Second-order regularity for the RTI linearization)**.** *On a neighborhood of the nominal RTI trajectory, $\mathbf{F}_\Theta$ is piecewise twice continuously differentiable with bounded second derivative, $\|\mathrm{D}^2\mathbf{F}_\Theta\|_2 \le M_{\mathrm{F}}$ wherever it exists, where $\|\cdot\|_2$ denotes the operator norm of the second-derivative bilinear form. The nominal point and its perturbation segment are assumed to meet finitely many smooth regions, so that the integral remainder below is well defined.*

Theorem 1 (Finite-Horizon Prediction Error Bound)**.** *For a common prescribed input sequence $\{\mathbf{u}_k, \dots, \mathbf{u}_{k+H-1}\}$, suppose that the reference and predicted trajectories remain in the operating set Ω over the horizon H. Under Assumptions 1 and 2, and with $e_k = \boldsymbol{\xi}_k^\star - \hat{\boldsymbol{\xi}}_k$,*

$$\|e_{k+H}\|_2 \le L_{\mathrm{F}}^H \|e_k\|_2 + \sum_{i=0}^{H-1} L_{\mathrm{F}}^{H-1-i}\, \delta_{k+i}^\star. \tag{38}$$

If in addition the reference discrepancy is uniformly bounded by ϵ as in Assumption 1, then

$$\|e_{k+H}\|_2 \le L_{\mathrm{F}}^H \|e_k\|_2 + \epsilon \sum_{j=0}^{H-1} L_{\mathrm{F}}^j. \tag{39}$$

If the horizon is initialized from the same measured history and the corresponding encoder-consistent latent state, i.e. $\hat{\mathbf{X}}_k = \mathbf{X}_k$ and $\mathbf{z}_k^P = g_\theta(\mathbf{X}_k)$, then $e_k = 0$ and $\|e_{k+H}\|_2 \le \epsilon \sum_{j=0}^{H-1} L_{\mathrm{F}}^j$. The bound compares the learned predictor with the recorded reference trajectory under a common *input sequence; it is not a closed-loop tracking-error or robustness statement. Here ϵ is the uniform bound of Assumption 1, not the trajectory-wise encoder-consistency defect ϵ_k of Proposition 2, nor the data-consistency residual $\mathbf{w}_k$ of Section IV-C.*

Proof. For one step, using the recorded successor $\boldsymbol{\xi}_{k+1}^\star$,

$$\begin{aligned}
\|e_{k+1}\|_2 &= \big\|\boldsymbol{\xi}_{k+1}^\star - \mathbf{F}_\Theta(\hat{\boldsymbol{\xi}}_k, \mathbf{u}_k)\big\|_2 \\
&\le \big\|\boldsymbol{\xi}_{k+1}^\star - \mathbf{F}_\Theta(\boldsymbol{\xi}_k^\star, \mathbf{u}_k)\big\|_2 \\
&\quad + \big\|\mathbf{F}_\Theta(\boldsymbol{\xi}_k^\star, \mathbf{u}_k) - \mathbf{F}_\Theta(\hat{\boldsymbol{\xi}}_k, \mathbf{u}_k)\big\|_2 \\
&\le \delta_k^\star + L_{\mathrm{F}} \|e_k\|_2,
\end{aligned}$$

by (35) and Assumption 2, which applies at every step because the trajectories remain in Ω over the horizon. Unrolling the recursion yields (38); bounding each $\delta_{k+i}^\star$ by ϵ yields (39). □

Remark 3 (ϵ, ϵ_k, and $\mathbf{w}_k$)**.** *The uniform bound ϵ in Theorem 1 and the encoder-consistency defect ϵ_k in Proposition 2 serve different purposes and should not be identified. The former bounds the one-step reference discrepancy $\delta_k^\star$ between the recorded reference augmented sequence and the learned transition $\mathbf{F}_\Theta$ over the operating set, whereas the latter measures the one-step discrepancy between the propagated and the encoder-consistent latent states along the predictor's own rollout. The data-consistency residual $\mathbf{w}_k$ of Section IV-C is a third quantity, evaluated on the recorded histories, and coincides with neither. Accordingly, the empirical statistics of ϵ_k are not used as a certified estimate of ϵ.*

Since the empirical Lipschitz constant satisfies $\widehat{L_{\mathrm{F}}} \approx 1.45 > 1$ (Section VII-B), we use only the finite-horizon form and do not invoke the asymptotic limit $\epsilon/(1 - L_{\mathrm{F}})$.

Corollary 1 (Physical-Space Prediction Bound)**.** *The current physical state $\mathbf{x}_{k+H}$ occupies the last n coordinates of the history block of $\boldsymbol{\xi}_{k+H}^\star$, and the predictor's decoded feedback $\hat{\mathbf{x}}_{k+H} = \mathbf{C}\mathbf{z}_{k+H}^P$ occupies the same coordinates of $\hat{\boldsymbol{\xi}}_{k+H}$. Since a coordinate sub-vector has at most the norm of the full vector and $\|\mathbf{C}\|_2 = 1$,*

$$\begin{aligned}
\|\hat{\mathbf{x}}_{k+H} - \mathbf{x}_{k+H}\|_2 &\le \|\hat{\boldsymbol{\xi}}_{k+H} - \boldsymbol{\xi}_{k+H}^\star\|_2 \\
&\le L_{\mathrm{F}}^H \|e_k\|_2 + \sum_{i=0}^{H-1} L_{\mathrm{F}}^{H-1-i}\, \delta_{k+i}^\star.
\end{aligned} \tag{40}$$

Proof. Both $\hat{\mathbf{x}}_{k+H}$ and $\mathbf{x}_{k+H}$ are the last n coordinates of the history block of the respective augmented states, so their difference is a coordinate sub-vector of $\hat{\boldsymbol{\xi}}_{k+H} - \boldsymbol{\xi}_{k+H}^\star$; hence $\|\hat{\mathbf{x}}_{k+H} - \mathbf{x}_{k+H}\|_2 \le \|\hat{\boldsymbol{\xi}}_{k+H} - \boldsymbol{\xi}_{k+H}^\star\|_2$. Substituting Theorem 1 yields (40). □

D. Local Linearization Remainder

The controller of Section VI linearizes the augmented model around a warm-start nominal trajectory. To write the second-order remainder unambiguously, we treat $\mathbf{F}_\Theta$ as a function of the joint variable $\eta = \mathrm{col}(\boldsymbol{\xi}, \mathbf{u})$, and let $\bar{\eta}_i = \mathrm{col}(\bar{\boldsymbol{\xi}}_i, \bar{\mathbf{u}}_i)$ and $\delta\eta_i = \mathrm{col}(\delta\boldsymbol{\xi}_i, \delta\mathbf{u}_i)$ denote the nominal point and the joint perturbation:

$$\mathbf{F}_\Theta(\bar{\eta}_i + \delta\eta_i) = \mathbf{F}_\Theta(\bar{\eta}_i) + \mathrm{D}\mathbf{F}_\Theta(\bar{\eta}_i)\,\delta\eta_i + \mathcal{R}_i, \tag{41}$$

where $\mathrm{D}\mathbf{F}_\Theta(\bar{\eta}_i)\,\delta\eta_i = \mathbf{F}_i\,\delta\boldsymbol{\xi}_i + \mathbf{G}_i\,\delta\mathbf{u}_i$ with $\mathbf{F}_i = \partial\mathbf{F}_\Theta/\partial\boldsymbol{\xi}$ and $\mathbf{G}_i = \partial\mathbf{F}_\Theta/\partial\mathbf{u}$. Since $\mathbf{F}_\Theta$ is twice continuously differentiable (Assumption 3), the second-order remainder admits the integral form

$$\mathcal{R}_i = \int_0^1 (1 - s)\, \mathrm{D}^2\mathbf{F}_\Theta\big(\bar{\eta}_i + s\delta\eta_i\big)\big[\delta\eta_i, \delta\eta_i\big]\, \mathrm{d}s, \tag{42}$$

where $\mathrm{D}^2\mathbf{F}_\Theta[\cdot,\cdot]$ denotes the second-derivative bilinear form.

Proposition 3 (Local Linearization Remainder Bound)**.** *Under Assumption 3,*

$$\|\mathcal{R}_i\|_2 \le \frac{M_{\mathrm{F}}}{2}\|\delta\eta_i\|_2^2 = \frac{M_{\mathrm{F}}}{2}\left\|\begin{bmatrix}\delta\boldsymbol{\xi}_i \\ \delta\mathbf{u}_i\end{bmatrix}\right\|_2^2. \tag{43}$$

Proof. Starting from the integral remainder (42), the norm of a vector integral is bounded by the integral of the norm, and the second-derivative bilinear form is bounded by its operator norm:

$$\begin{aligned}\|\mathcal{R}_i\|_2 &\le \int_0^1 (1-s)\left\|\mathrm{D}^2\mathbf{F}_\Theta(\bar\eta_i + s\delta\eta_i)\right\|_2 \|\delta\eta_i\|_2^2\,\mathrm{d}s \\ &\le \frac{M_\mathrm{F}}{2}\|\delta\eta_i\|_2^2,\end{aligned}$$

where $\int_0^1(1-s)\,\mathrm{d}s = \frac{1}{2}$ and $\|\mathrm{D}^2\mathbf{F}_\Theta\|_2 \le M_\mathrm{F}$ on the line segment $\{\bar\eta_i + s\delta\eta_i : \ s \in [0,1]\}$ by Assumption 3. This is (43). □

This result establishes the local second-order accuracy of the linearized prediction model used in the RTI step; the practical adequacy of a single iteration is evaluated against the matched exact scheduled NMPC oracle in Section VI, and it is not a new SQP convergence theory.

Remark 4 (Scope of the analysis)**.** *The analysis is finite-horizon and concerns predictor consistency and local linearization accuracy. It does not establish global closed-loop stability, a common Lyapunov function, terminal invariant sets, or recursive feasibility under learned uncertainty. The regularity constants used for the empirical assessment are operating-region diagnostics rather than global certificates, all bounds concern the rolled-out latent predictor rather than true-plant tracking error, and* $\mathbf{A}(\mathbf{c})$ *is a latent linearization parameterization rather than an exact Koopman invariant-subspace operator. The rollout objective* $\mathcal{L}_R$ *of Section IV-B is likewise an algorithmic training mechanism: minimizing it is not claimed to imply any bound on* ϵ_k*,* $d_\mathcal{M}(H)$*, or* L_F*.*

VI. Real-Time Predictive Control

We integrate the rollout-trained scheduled predictor into a receding-horizon controller that runs within the sampling period $T_s = 200\,\mathrm{ms}$ of the soft-robot platform.

A. History-Consistent Scheduled Prediction

At each control instant the controller predicts over a horizon N by propagating the latent and re-scheduling the context from the decoded history,

$$\mathbf{z}_{i+1} = \mathbf{A}(\mathbf{c}_i)\mathbf{z}_i + \mathbf{B}\mathbf{u}_i, \quad \mathbf{c}_i = C_\phi(\hat{\mathbf{X}}_i, \mathbf{U}_i^\mathrm{in}), \quad \hat{\mathbf{x}}_i = \mathbf{C}\mathbf{z}_i, \tag{44}$$

where $\hat{\mathbf{X}}_i$ is the rolled-out history. This *context-re-scheduled, latent-propagated* structure (RTI-SP) never re-runs the full Transformer lifting encoder inside the horizon. The alternative—re-encoding the history at every prediction stage (RTI-RE)—is more accurate in principle but costs one encoder evaluation per stage.

B. One-Iteration RTI

We solve the receding-horizon problem

$$\min_{\mathbf{u}_{0:N-1}} \sum_{i=1}^{N} \|\hat{\mathbf{x}}_i - \mathbf{x}_i^\mathrm{ref}\|_\mathbf{Q}^2 + \sum_{i=0}^{N-1}\left(\|\mathbf{u}_i\|_\mathbf{R}^2 + \|\mathbf{u}_i - \mathbf{u}_{i-1}\|_\mathbf{S}^2\right) \tag{45}$$

subject to $\mathbf{u}_i \in [\mathbf{u}_\min, \mathbf{u}_\max]$, using a single real-time-iteration (RTI) SQP step per sampling instant [43], [44]: (i) warm-start from the shifted previous solution; (ii) roll out the nominal trajectory (44); (iii) linearize to obtain $\delta\boldsymbol{\xi}_{i+1} = \mathbf{F}_i\delta\boldsymbol{\xi}_i + \mathbf{G}_i\delta\mathbf{u}_i$; (iv) solve one quadratic program over $\delta\mathbf{u}_{0:N-1}$; and (v) apply only the first input. Proposition 3 bounds the second-order linearization remainder of the model used in the RTI step; the adequacy of a single iteration is assessed empirically against the matched exact scheduled NMPC oracle, and we make no global SQP convergence claim.

TABLE I
One-iteration RTI on the soft-robot proxy ($N = 5$). The matched oracle is an exact scheduled NMPC solved to convergence at the same horizon and plant.

Controller	RMSE	Δ vs oracle	p50 / p95 (ms)
Exact scheduled oracle	0.1635	—	1465 / 1931
RTI-SP (proposed)	0.1618	−1.04%	**78 / 120**
Exact re-encode oracle	0.1582	—	1958 / 2930
RTI-RE	0.1565	−1.05%	182 / 253

C. Real-Time Behavior and Fidelity

Table I reports the soft-robot closed-loop proxy (plant: an independently trained direct neural predictor; recorded validation references; $N = 5$).

Three points follow. First, one-iteration RTI-SP tracks the *matched* exact scheduled oracle to within about 1% at $N = 5$, i.e. the RTI approximation introduces no meaningful tracking degradation. Second, RTI-SP is roughly 2.3× faster than RTI-RE at the median and 2.1× at $p95$, because it avoids a full Transformer lifting-encoder evaluation per stage, and its $p95$ latency (120 ms) is well below the $\sim$ 200 ms soft-robot sampling period. Third, the small negative Δ should not be read as RTI outperforming exact NMPC; it is attributable to numerical optimization, warm-starting, finite simulation length, solver tolerance, and receding-horizon trajectory differences. Accordingly, the exact oracle is not a strict RMSE upper bound, which is why Proposition 3 is assessed by its own remainder diagnostic (Section VII-B) rather than by this comparison. We note that a longer horizon ($N = 8$) tracks slightly better but exceeds the sampling period ($p95 = 215$ ms) and is therefore not used.

VII. Experiments

This section reports a training-schedule ablation of the history-conditioned predictor, followed by the empirical diagnostics that assess the assumptions and quantities of the consistency analysis of Section V.

A. Training Schedule Ablation

We assess how the training schedule affects the deployment accuracy of the history-conditioned predictor. All variants share the same architecture, data, optimizer, rollout horizon ($H_R = 20$), learning rate, batch size, and scheduled-rollout-stage epoch budget. They differ in which parameters the scheduled-rollout stage optimizes: strategy A optimizes only

TABLE II
TRAINING-SCHEDULE ABLATION: VALIDATION ROLLOUT RMSE UNDER SCHEDULED PROPAGATION, AVERAGED OVER THREE SEEDS ($H_R = 20$ DURING TRAINING). THE STANDARD DEVIATION AT $H = 20$ IS AT MOST 0.0011 FOR B–D.

	Scheduled-rollout stage	$H = 1$	$H = 5$	$H = 10$	$H = 20$
A	modulation only	0.0276	0.1787	0.4300	1.1533
B	all but $\mathbf{B}$	0.0173	0.0271	0.0337	0.0440
C	all parameters	0.0115	0.0228	0.0301	0.0415
D	all, no pretraining	0.0112	0.0217	0.0289	0.0401

the low-rank modulation $(\phi, \mathbf{U}, \mathbf{V}, \gamma_\psi)$ while freezing the temporal lifting θ, the nominal transition $\mathbf{A}_0$, and the input matrix $\mathbf{B}$; strategy B freezes only $\mathbf{B}$; strategy C optimizes all parameters; strategy D additionally omits the nominal one-step pretraining stage and optimizes all parameters from a random initialization. Table II reports the validation rollout RMSE under the deployment-consistent scheduled propagation, averaged over three seeds.

Three observations follow. First, what matters is which parameters the scheduled-rollout objective is allowed to adapt: optimizing only the low-rank modulation while freezing the temporal lifting and the nominal transition (A) degrades the $H = 20$ rollout error by more than an order of magnitude (1.1533 versus 0.0415), so the representation and the nominal transition must adapt together with the modulation. Second, freezing the input matrix brings no benefit: variant B is worse than variant C at every horizon and in every seed, with a 5.9% higher $H = 20$ rollout RMSE and a 24% higher rollout MSE accumulated over the whole 20-step rollout. Third, the nominal pretraining stage is not required: optimizing all parameters directly under the scheduled-rollout objective (D) matches or slightly exceeds C while removing one training stage and reducing the total training time (311 s versus 338 s). We therefore treat the two-stage schedule of Section IV as an optimization initialization rather than a necessity of the model. All remaining results use the deployment predictor described in Section IV, which was trained with the same schedule as variant C; the ablation variants isolate the contribution of each element of that schedule. The separation of B from C and D exceeds the seed-to-seed standard deviation, whereas C and D are statistically indistinguishable.

We also verify that the deployment failure is not an artifact of unbounded latent dynamics. Training with an explicit spectral constraint $\rho(\mathbf{A}(\mathbf{c})) \leq 0.99$ under the one-step objective attains an $H = 20$ rollout RMSE of 1.5681, whereas the same constrained model trained under the scheduled-rollout objective attains 0.0412 (both single-seed comparisons). Enforcing spectral stability therefore does not repair the failure, whereas changing the training objective does.

B. Mechanism and Theory Diagnostics

Table III reports the diagnostics measured on the frozen deployment predictor of Section IV (all parameters optimized under the scheduled-rollout objective, $r = 2$, scheduled propagation) on the soft-robot validation set ($P = 8$, $n = 2$, $m = 2$, $d = 10$). These diagnostics are empirical measurements on the held-out validation distribution: they assess the consistency of the theoretical assumptions and quantities with the operating data, rather than certify global worst-case constants over the entire operating set Ω.[1]

TABLE III
THEORY DIAGNOSTICS ON THE FROZEN SCHEDULED-PROPAGATION PREDICTOR.

Quantity	Role	Measured ($H = 1/5/10/20$)
ϵ_k (absolute)	Prop. 2	median 0.345; q95 0.567
$d^{\mathrm{abs}}(H)$	Prop. 2	0.346/1.122/1.645/2.663
$d_{\mathcal{M}}(H)$	Def. 3	0.120/0.390/0.574/0.928
$\|\mathbf{A}_{k+H-1} \cdots \mathbf{A}_k\|_2$	Prop. 2	1.046/1.247/1.541/2.304
$\|\partial \mathbf{F}_\Theta / \partial \boldsymbol{\xi}\|_2$ ($\widehat{L_{\mathrm{F}}}$)	Assumption 2	median 1.445; max 1.452
Prop. 2 bound / actual	sanity	1.64/2.77/4.22/6.52×
$\widehat{M_{\mathrm{F}}}$ (fitted, δ^2-scaled)	Prop. 3	≈ 0.007
physical RMSE	Cor. 1	0.011/0.022/0.030/0.042
RTI runtime	deployment	p50 78 / p95 120 ms

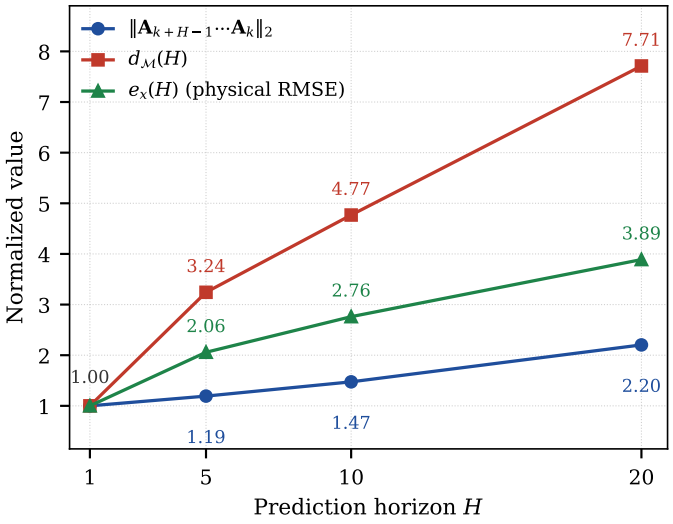


Fig. 3. Normalized growth with the prediction horizon H of the finite transition-product norm $\|\mathbf{A}_{k+H-1} \cdots \mathbf{A}_k\|_2$, the latent rollout defect $d_{\mathcal{M}}(H)$, and the physical prediction error $e_x(H)$; each curve is divided by its value at $H = 1$ so that different units share one axis. The three quantities grow together with H, which is *consistent with* the error-propagation mechanism of Proposition 2. The figure is an empirical illustration and does not by itself establish a causal chain.

Three observations follow, illustrated in Fig. 3. First, the transition-product norm grows from 1.05 to 2.30 (no contraction) in step with $d_{\mathcal{M}}$, numerically supporting Remark 2. Second, the empirical Lipschitz constant is $\widehat{L_{\mathrm{F}}} \approx 1.45 > 1$, so the finite-horizon form of Theorem 1 is the applicable statement and the infinite-horizon limit $\epsilon/(1 - L_{\mathrm{F}})$ is not available. Third, the Proposition 2 bound is always $\geq$ the actual defect and only 1.64–6.52× conservative when compared like-for-like (absolute one-step defects ϵ_k against absolute rollout defects d^{abs}), and the fitted $\widehat{M_{\mathrm{F}}} \approx 0.007$ is stable across perturbation scales, supporting Proposition 3. The physical-space bound of Corollary 1 is valid but very loose (using

[1] ϵ_k and d^{abs} are evaluated on all admissible validation windows (stride one); the H-step rollout statistics $d_{\mathcal{M}}$ and the transition-product norms use a stride-four subsample of the same trajectories. All values are medians unless a quantile is stated.

$\widehat{L_{\mathrm{F}}} = 1.445$, 53–47 750× over $H = 1, \ldots, 20$), because $\widehat{L_{\mathrm{F}}}^{H}$ grows geometrically while the true error saturates; we report this honestly rather than presenting the bound as tight.

These measurements bear on the claim that one-step training insufficiently constrains the rollout distribution. That claim rests on the *conjunction* of four observations—a positive one-step consistency defect ($\epsilon_k > 0$), a rollout defect $d_{\mathcal{M}}(H)$ that grows with the horizon, the failure of naive latent propagation, and the recovery obtained under rollout-aware training—and no single observation is sufficient on its own; the latter two are quantified in the experimental study.

VIII. Conclusion

This paper presented Transformer-Koopman, a framework for learning history-conditioned Koopman models of controlled nonlinear systems from finite history windows. Because the measured state need not reveal the system's internal dynamics, a patch-based Transformer encoder first maps the history to a structured, state-preserving lifted state. Because a single global lifted transition would then still require all history-dependent regimes to share one evolution law, the lifted transition is additionally modulated by the history through a rank-r term $\mathbf{A}(\mathbf{c}) = \mathbf{A}_0 + \mathbf{U}\,\mathrm{diag}(\gamma_\psi(\mathbf{c}))\mathbf{V}^\top$ ($r = 2$), which confines history dependence to a structured correction of a shared nominal operator while keeping the propagation conditional-linear. We identified a training–deployment inconsistency between one-step-trained history-conditioned dynamics and the recursively generated latent states seen at deployment, showed that it is not repaired by spectral stabilization, and resolved it with deployment-consistent scheduled training. The rollout-trained scheduled predictor was integrated into a one-iteration RTI-MPC that runs in real time and matches a matched exact scheduled NMPC oracle to within about 1%. A finite-horizon theory—finite-history Markovization, latent rollout error propagation, a bounded prediction-error theorem, a physical-space corollary, and a local RTI approximation result—was established and supported by empirical diagnostics. Hardware validation on the soft-robot platform across multi-setpoint, circular, figure-eight, unseen-trajectory, and speed-shift regimes, together with closed-loop baselines against a fixed (time-invariant) Transformer-Koopman controller, remains the immediate next step.

References

[1] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Natl. Acad. Sci. USA*, vol. 17, no. 5, pp. 315–318, 1931.

[2] B. O. Koopman and J. von Neumann, "Dynamical systems of continuous spectra," *Proc. Natl. Acad. Sci. USA*, vol. 18, no. 3, pp. 255–263, 1932.

[3] I. Mezić, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dyn.*, vol. 41, no. 1–3, pp. 309–325, 2005.

[4] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 1999.

[5] P. Van Overschee and B. De Moor, "N4SID: Subspace algorithms for the identification of combined deterministic-stochastic systems," *Automatica*, vol. 30, no. 1, pp. 75–93, 1994.

[6] O. Nelles, *Nonlinear System Identification: From Classical Approaches to Neural Networks, Fuzzy Models, and Gaussian Processes*, 2nd ed. Cham, Switzerland: Springer, 2020.

[7] C. W. Rowley, I. Mezić, S. Bagheri, P. Schlatter, and D. S. Henningson, "Spectral analysis of nonlinear flows," *J. Fluid Mech.*, vol. 641, pp. 115–127, 2009.

[8] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data–driven approximation of the Koopman operator: Extending dynamic mode decomposition," *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.

[9] M. O. Williams, C. W. Rowley, and I. G. Kevrekidis, "A kernel-based method for data-driven Koopman spectral analysis," *J. Comput. Dyn.*, vol. 2, no. 2, pp. 247–265, 2015.

[10] H. Arbabi and I. Mezić, "Ergodic theory, dynamic mode decomposition, and computation of spectral properties of the Koopman operator," *SIAM J. Appl. Dyn. Syst.*, vol. 16, no. 4, pp. 2096–2126, 2017.

[11] M. Kamb, E. Kaiser, S. L. Brunton, and J. N. Kutz, "Time-delay observables for Koopman: Theory and applications," *SIAM J. Appl. Dyn. Syst.*, vol. 19, no. 2, pp. 886–917, 2020.

[12] S. L. Brunton, B. W. Brunton, J. L. Proctor, E. Kaiser, and J. N. Kutz, "Chaos as an intermittently forced linear system," *Nat. Commun.*, vol. 8, art. 19, 2017.

[13] S. Das and D. Giannakis, "Delay-coordinate maps and the spectra of Koopman operators," *J. Stat. Phys.*, vol. 175, no. 6, pp. 1107–1145, 2019.

[14] Y. Liu, C. Li, J. Wang, and M. Long, "Koopa: Learning non-stationary time series dynamics with Koopman predictors," in *Proc. NeurIPS*, 2023.

[15] Z. Li, M. Han, and X. Yin, "MamKO: Mamba-based Koopman operator for modeling and predictive control," in *Proc. ICLR*, 2025.

[16] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, "Modern Koopman theory for dynamical systems," *SIAM Rev.*, vol. 64, no. 2, pp. 229–340, 2022.

[17] S. E. Otto and C. W. Rowley, "Koopman operators for estimation and control of dynamical systems," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 4, no. 1, pp. 59–87, 2021.

[18] N. Takeishi, Y. Kawahara, and T. Yairi, "Learning Koopman invariant subspaces for dynamic mode decomposition," in *Proc. NeurIPS*, 2017, pp. 1130–1140.

[19] E. Yeung, S. Kundu, and N. Hodas, "Learning deep neural network representations for Koopman operators of nonlinear dynamical systems," in *Proc. ACC*, 2019, pp. 4832–4839.

[20] B. Lusch, J. N. Kutz, and S. L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nat. Commun.*, vol. 9, no. 1, p. 4950, 2018.

[21] J. Morton, F. D. Witherden, and M. J. Kochenderfer, "Deep variational Koopman models: Inferring Koopman observations for uncertainty-aware dynamics modeling and control," in *Proc. IJCAI*, 2019, pp. 3173–3179.

[22] O. Azencot, N. B. Erichson, V. Lin, and M. W. Mahoney, "Forecasting sequential data using consistent Koopman autoencoders," in *Proc. ICML*, 2020, pp. 475–485.

[23] L. Wu, W. G. Y. Tan, L. Zhou, R. D. Braatz, and J. Drgoňa, "Least-squares multi-step Koopman operator learning for model predictive control," arXiv:2601.11901, 2026.

[24] S. E. Otto and C. W. Rowley, "Linearly-recurrent autoencoder networks for learning dynamics," *SIAM J. Appl. Dyn. Syst.*, vol. 18, no. 1, pp. 558–593, 2019.

[25] M. Forgione and D. Piga, "dynoNet: A neural network architecture for learning dynamical systems," *Int. J. Adapt. Control Signal Process.*, vol. 35, no. 4, pp. 612–626, 2021.

[26] J. L. Proctor, S. L. Brunton, and J. N. Kutz, "Dynamic mode decomposition with control," *SIAM J. Appl. Dyn. Syst.*, vol. 15, no. 1, pp. 142–161, 2016.

[27] J. L. Proctor, S. L. Brunton, and J. N. Kutz, "Generalizing Koopman theory to allow for inputs and control," *SIAM J. Appl. Dyn. Syst.*, vol. 17, no. 1, pp. 909–930, 2018.

[28] S. L. Brunton, B. W. Brunton, J. L. Proctor, and J. N. Kutz, "Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control," *PLOS ONE*, vol. 11, no. 2, e0150171, 2016.

[29] M. Korda and I. Mezić, "Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control," *Automatica*, vol. 93, pp. 149–160, 2018.

[30] Y. Han, W. Hao, and U. Vaidya, "Deep learning of Koopman representation for control," in *Proc. IEEE Conf. Decision Control (CDC)*, 2020, pp. 1890–1895.

[31] H. Shi and M. Q.-H. Meng, "Deep Koopman operator with control for nonlinear systems," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7700–7707, 2022.

[32] R. Strässer, K. Worthmann, I. Mezić, J. Berberich, M. Schaller, and F. Allgöwer, "An overview of Koopman-based control: From error bounds to closed-loop guarantees," *Annu. Rev. Control*, vol. 61, art. 101035, 2026.
[33] J. F. Durán-Sigüenza, L. I. Minchala, L. E. Garza-Castañón, and H. Zhang, "Control based on the Koopman operator: A comprehensive review," *J. Franklin Inst.*, vol. 362, no. 18, art. 108256, 2025.
[34] S. Klus, F. Nüske, P. Koltai, H. Wu, I. Kevrekidis, C. Schütte, and F. Noé, "Data-driven model reduction and transfer operator approximation," *J. Nonlinear Sci.*, vol. 28, no. 3, pp. 985–1010, 2018.
[35] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.
[36] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with Transformers," in *Proc. ICLR*, 2023.
[37] A. Forootani, M. Khosravi, and M. Barati, "DeepKoopFormer: A Koopman enhanced Transformer based architecture for time series forecasting," arXiv:2508.02616, 2025.
[38] A. Forootani and R. Iervolino, "Learnable Koopman-enhanced Transformer-based time series forecasting with spectral control," arXiv:2602.02592, 2026.
[39] N. Feizi, F. C. Pedrosa, J. Jayender, and R. V. Patel, "Deep Koopman approach for nonlinear dynamics and control of tendon-driven continuum robots," *IEEE Robot. Autom. Lett.*, vol. 10, no. 3, pp. 2926–2933, 2025.
[40] G.-C. Rota and G. Strang, "A note on the joint spectral radius," *Nederl. Akad. Wetensch. Proc. Ser. A*, vol. 63 (Indag. Math., vol. 22), pp. 379–381, 1960.
[41] I. Daubechies and J. C. Lagarias, "Sets of matrices all infinite products of which converge," *Linear Algebra Appl.*, vol. 161, pp. 227–263, 1992.
[42] R. M. Jungers, *The Joint Spectral Radius: Theory and Applications*, Lecture Notes in Control and Information Sciences, vol. 385. Berlin, Germany: Springer, 2009.
[43] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," *SIAM J. Control Optim.*, vol. 43, no. 5, pp. 1714–1736, 2005.
[44] S. Gros, M. Zanon, R. Quirynen, A. Bemporad, and M. Diehl, "From linear to nonlinear MPC: Bridging the gap via the real-time iteration," *Int. J. Control*, vol. 93, no. 1, pp. 62–80, 2020.
[45] M. Diehl, R. Findeisen, F. Allgöwer, H. G. Bock, and J. P. Schlöder, "Nominal stability of real-time iteration scheme for nonlinear model predictive control," *IEE Proc. Control Theory Appl.*, vol. 152, no. 3, pp. 296–308, 2005.
[46] V. M. Zavala and L. T. Biegler, "The advanced-step NMPC controller: Optimality, stability and robustness," *Automatica*, vol. 45, no. 1, pp. 86–93, 2009.
[47] W. J. Rugh and J. S. Shamma, "Research on gain scheduling," *Automatica*, vol. 36, no. 10, pp. 1401–1425, 2000.
[48] D. J. Leith and W. E. Leithead, "Gain-scheduled and nonlinear systems: Dynamic analysis by velocity-based linearization families," *Int. J. Control*, vol. 70, no. 2, pp. 289–317, 1998.
[49] P. S. G. Cisneros, S. Voss, and H. Werner, "Efficient nonlinear model predictive control via quasi-LPV representation," in *Proc. 55th IEEE Conf. Decision Control (CDC)*, 2016, pp. 3216–3221.
[50] P. S. G. Cisneros and H. Werner, "Nonlinear model predictive control for models in quasi-linear parameter varying form," *Int. J. Robust Nonlinear Control*, vol. 30, no. 10, pp. 3945–3959, 2020.